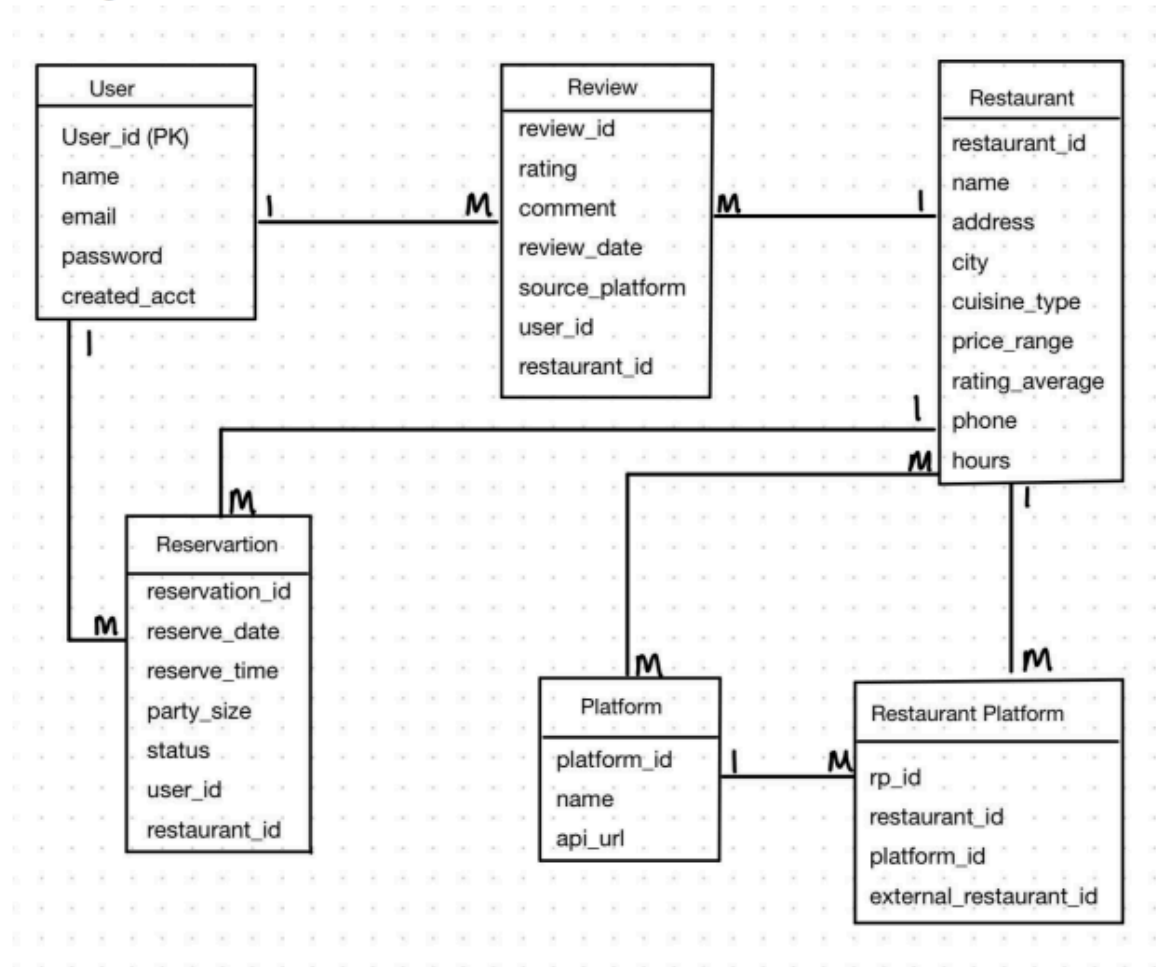


Members: Kowan Atcitty, Khoa Vo, Colton Jim

ER Diagram



Part 1: ER-to-Relational (4 pt): Transform the ER diagram into relational model using SQL data definition language (DDL)

Our Project Description: Our project will be centered around restaurant booking and reviews. Where the users would be diners with attributes that include their booking information (ex: name, email, and phone number). Other entities in this database would be restaurants, reviews, reservations, and platforms. Each with their own personal attributes. The user within this application would be able to interact and browse through various types of restaurants, as well be able to pull reviews from other platforms (ie: Yelp and Google).

We transformed each core entity in the ER design into a relational table in PostgreSQL

DROP TABLE IF EXISTS restaurant_platforms CASCADE;

DROP TABLE IF EXISTS reviews CASCADE;

DROP TABLE IF EXISTS reservations CASCADE;

DROP TABLE IF EXISTS platforms CASCADE;

DROP TABLE IF EXISTS restaurants CASCADE;

DROP TABLE IF EXISTS users CASCADE;

```
CREATE TABLE users (  
    user_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    phone VARCHAR(20),  
    password_hash VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE restaurants (  
    restaurant_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR(150) NOT NULL,  
    address VARCHAR(200) NOT NULL,  
    city VARCHAR(100) NOT NULL,  
    cuisine_type VARCHAR(100),  
    price_range VARCHAR(20),  
    phone VARCHAR(20),  
    hours VARCHAR(100)  
);
```

```
CREATE TABLE platforms (  
    platform_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    platform_name VARCHAR(100) NOT NULL UNIQUE,  
    api_url VARCHAR(255)  
);
```

```
CREATE TABLE reviews (  
    review_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    user_id INT NOT NULL,  
    restaurant_id INT NOT NULL,  
    rating NUMERIC(2,1) NOT NULL CHECK (rating >= 1 AND rating <= 5),  
    comment TEXT,  
    review_date DATE NOT NULL,  
    source_platform VARCHAR(50),  
    FOREIGN KEY (user_id) REFERENCES users(user_id)  
        ON DELETE CASCADE,  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
        ON DELETE CASCADE  
);
```

```
CREATE TABLE reservations (  
    reservation_id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    restaurant_id INT NOT NULL,  
    reservation_date DATE NOT NULL,  
    reservation_time TIME NOT NULL,  
    party_size INT NOT NULL CHECK (party_size > 0),  
    status VARCHAR(20) NOT NULL CHECK (status IN ('pending', 'confirmed',  
'cancelled', 'completed')),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(user_id)  
        ON DELETE CASCADE,  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
        ON DELETE CASCADE  
);
```

```
CREATE TABLE restaurant_platforms (  
    restaurant_id INT NOT NULL,  
    platform_id INT NOT NULL,  
    external_restaurant_id VARCHAR(100) NOT NULL,  
    PRIMARY KEY (restaurant_id, platform_id),  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
        ON DELETE CASCADE,  
    FOREIGN KEY (platform_id) REFERENCES platforms(platform_id)  
        ON DELETE CASCADE  
);
```

Part 2: *Fill in the database with data* (synthetically generated, scraped from web data or integrated from open data) (4 pt): . That should be done through a sequence of SQL Insert/Copy statements.

Here I developed six different groups of insert statements for six different tables. The **users** table adds people who use the application and stores their respective parameters. The **restaurants** table adds restaurants into the database and stores the attributes of a restaurant. The **platforms** table stores the external platforms of where the reviews come from. In the platform table yelp and google are the external sources used. The **reviews** table stores reviews inputted by a user. The **reservations** table stores the table booking by a user. The last one, **restaurant_platforms** table connects restaurants to the different platforms which are Yelp and Google.

```
INSERT INTO users (name, email, phone, password_hash)
VALUES
('Colton Jim', 'asurite1@asu.edu', '1234567890', 'hash1'),
('Khoa Vo', 'asurite2@asu.edu', '2345678901', 'hash2'),
('Kowan Atcitty', 'asurite3@asu.edu', '3456789012', 'hash3');
```

```
INSERT INTO restaurants (name, address, city, cuisine_type, price_range, phone, hours)
VALUES
('Pitch Forks', '123 Main St', 'Tempe', 'Variety of cuisines', '$$', '4801111111', '10:00 AM - 9:00 PM'),
('Sushi Palace', '456 Elm St', 'Phoenix', 'Japanese', '$$$', '4802222222', '11:00 AM - 10:00 PM'),
('Chipotle', '789 Oak St', 'Tempe', 'Mexican', '$', '4803333333', '9:00 AM - 11:00 PM');
```

```
INSERT INTO platforms (platform_name, api_url)
VALUES
('Yelp', 'https://api.yelp.com'),
('Google', 'https://maps.googleapis.com');
```

```
INSERT INTO reviews (user_id, restaurant_id, rating, comment, review_date, source_platform)
VALUES
(1, 1, 4.5, 'Dining hall food was great!', '2026-03-01', 'Yelp'),
(2, 2, 5.0, 'Amazing sushi!', '2026-03-02', 'Google');
```

```
INSERT INTO reservations (reservation_id, user_id, restaurant_id, reservation_date,
reservation_time, party_size, status, created_at)
VALUES
(1, 1, 1, '2026-04-01', '19:00', 4, 'confirmed', '2026-04-01'),
(2, 2, 2, '2026-04-02', '18:30', 2, 'pending', '2026-04-02');
```

```
INSERT INTO restaurant_platforms (restaurant_id, platform_id, external_restaurant_id)
VALUES
(1, 1, 'yelp_001'),
(2, 2, 'google_002');
```

3. *SQL Queries* (4 pt): Prepare examples of SQL queries that cover the application description (as described by the proposal submitted). The SQL queries may include SELECT queries or INSERT/UPDATE/DELETE queries.

Select query

```
SELECT * FROM users;
SELECT * FROM restaurants;
SELECT * FROM platforms;
SELECT * FROM reviews;
SELECT * FROM reservations;
SELECT * FROM restaurant_platforms;
```

```
SELECT name, city, cuisine_type
FROM restaurants
WHERE city = 'Tempe';
```

```
SELECT review_id, restaurant_id, rating, comment
FROM reviews
WHERE rating >= 4.5;
```

Insert query

Insert a new user:

```
INSERT INTO users (name, email, phone, password_hash)
VALUES
('student', 'student@asu.edu', '6890123456', 'hash4');
```

Insert a new restaurant:

```
INSERT INTO restaurants (name, address, city, cuisine_type, price_range, phone, hours)
VALUES
('Barrett Dining Hall', '821 East Lemon Hall', 'Tempe', 'Variety of cuisines', '$$', '4804444444',
'9:00 AM - 9:00 PM');
```

Make a review:

```
INSERT INTO reviews (user_id, restaurant_id, rating, comment, review_date, source_platform)
VALUES
(1, 4, 3.0, 'It was meh', '2026-03-25', 'Google');
```

Make a new reservation:

```
INSERT INTO reservations
(reservation_id, user_id, restaurant_id, reservation_date, reservation_time, party_size, status)
VALUES((SELECT COALESCE(MAX(reservation_id), 0) + 1 FROM reservations), 4, 4,
'2026-04-01', '18:00', 1, 'confirmed');
```

Update query

Update a review:

```
UPDATE reviews SET rating = 5, comment = 'updated review' WHERE review_id = 1;
```

Update reservation:

```
UPDATE reservations SET reservation_date = '2026-04-02', reservation_time = '18:00',
party_size = 6 WHERE reservation_id = 1;
```

Delete query

Delete user 1:

```
DELETE FROM users WHERE user_id = 4;
```

Delete restaurant 2 :

```
DELETE FROM restaurants WHERE restaurant_id = 4;
```

^^also deletes the reviews and reservations made for the restaurant as well

Delete review 3:

```
DELETE FROM reviews WHERE review_id = 2;
```

Delete reservation:

```
DELETE FROM reservations WHERE reservation_id = 1;
```

